

Phase 5: Quick Sort Partitioning — Full Structured Beginner Note

1. Current Progress Before Phase 5

Before Phase 5, you have already studied four important phases.

Phase 1: Sorting Foundations

In Phase 1, you learned the basic language used to understand sorting algorithms.

You learned words like:

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking two values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already or nearly sorted
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
$O(1)$	Constant time or constant space
$O(n)$	Linear time
$O(n^2)$	Quadratic time
$O(n \log n)$	Usually a fast comparison-sorting class
Pointer	A variable that stores an address

The most important lesson from Phase 1 was:

Do not judge a sorting algorithm only by asking, "Does it sort?"

A better question is:

How does it sort, how much time does it take, how much memory does it use, and does it preserve duplicate order?

Phase 2: Bubble Sort

In Phase 2, you learned Bubble Sort.

Bubble Sort works by comparing neighboring elements.

Example:

8 5

Since $8 > 5$, Bubble Sort swaps them:

5 8

Important points from Bubble Sort:

- It compares adjacent elements.
 - It swaps values when they are in the wrong order.
 - One full scan is called a pass.
 - After each pass, one large value is fixed at the end.
 - After k passes, the k largest values are fixed at the end.
 - It can be adaptive if we use a `flag`.
 - It is stable if we use $>$ instead of $>=$.
 - Best-case time is $O(n)$ when the adaptive flag is used.
 - Worst-case time is $O(n^2)$.
 - Space complexity is $O(1)$.
-

Phase 3: Insertion Sort

In Phase 3, you learned Insertion Sort.

Insertion Sort works by taking one value and inserting it into the correct position inside a sorted left part.

Example:

[5 8] 7 3 2

The left part [5 8] is sorted.

Now we insert 7 into the correct position:

[5 7 8] 3 2

Important points from Insertion Sort:

- It builds a sorted left part.
- It starts from index 1 because the first element alone is already sorted.
- It stores the selected value in `x`.
- It shifts bigger values to the right.
- It inserts `x` at the correct position.
- It is naturally adaptive.
- It is stable if we use `>` instead of `>=`.
- Best-case time is $O(n)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.
- It is good for nearly sorted data.

Phase 4: Selection Sort

In Phase 4, you learned Selection Sort.

Selection Sort works by selecting the smallest value from the unsorted part and putting it at the front.

Example:

8 6 3 2 5 4

The smallest value is 2, so Selection Sort swaps 2 with the first value:

2 6 3 8 5 4

Important points from Selection Sort:

- It selects the minimum value.
- It fills the array from left to right.
- It uses `i`, `j`, and `k`.

- `i` is the position being filled.
- `j` scans the unsorted part.
- `k` stores the index of the smallest value found so far.
- It swaps only after the inner scan finishes.
- It performs fewer swaps than Bubble Sort.
- It is not adaptive.
- It is usually not stable.
- Best-case time is $O(n^2)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.
- After `k` passes, the `k` smallest values are fixed at the front.

2. Why Phase 5 Comes After Selection Sort

Bubble Sort, Insertion Sort, and Selection Sort are simple comparison-based sorting algorithms.

They are useful for learning, but their worst-case time is usually $O(n^2)$.

Phase 5 begins a more powerful sorting idea: **Quick Sort**.

But Quick Sort can be difficult for beginners if we try to learn everything at once.

So the curriculum splits Quick Sort into two parts:

Phase	Topic	Main focus
Phase 5	Quick Sort Partitioning	Learn how to place one pivot in its correct position
Phase 6	Quick Sort Recursion and Complexity	Use recursion to sort the left and right parts

So in Phase 5, we are **not learning full Quick Sort yet**.

We are learning the most important part of Quick Sort:

Partitioning.

3. Main Goal of Phase 5

The main goal of Phase 5 is to understand this idea:

Choose one value, called the pivot, and place it in its final sorted position.

After partitioning, the pivot should be in a position where:

all smaller values are on the left
all greater values are on the right

This does not mean the whole array is sorted.

It only means the pivot is in its correct final place.

4. Quick Sort vs Selection Sort Idea

Selection Sort and Quick Sort both care about correct positions, but they think differently.

Selection Sort

Selection Sort chooses a **position** first.

Then it asks:

Which value belongs in this position?

Example:

8 6 3 2 5 4

For index **0**, Selection Sort finds the smallest value, which is **2**, and places it at index **0**.

So Selection Sort is like:

Choose position → find correct value

Quick Sort Partitioning

Quick Sort chooses a **value** first.

That value is called the pivot.

Then it asks:

Where does this pivot belong?

Example:

50 70 60 90 40 80 10 20 30

The pivot is 50.

Quick Sort partitioning finds the correct position for 50.

So Quick Sort partitioning is like:

Choose value → find correct position

5. What Is Quick Sort Partitioning?

Partitioning means dividing the array around a pivot.

After partitioning:

- Values smaller than the pivot should be on the left.
- Values greater than the pivot should be on the right.
- The pivot should be placed between them.

Example:

Before partitioning:
50 70 60 90 40 80 10 20 30

Pivot:

50

After partitioning:

40 30 20 10 | 50 | 80 90 60 70

The pivot 50 is now in its final sorted position.

Important:

```
40 30 20 10
```

is not fully sorted.

Also:

```
80 90 60 70
```

is not fully sorted.

But that is okay.

Partitioning only fixes the pivot.

6. What Does “Final Sorted Position” Mean?

An element is in its final sorted position when every value on its left is smaller and every value on its right is greater.

Suppose the pivot is 50.

If the array becomes:

```
40 30 20 10 50 80 90 60 70
```

Then 50 is in its final sorted position because:

```
40, 30, 20, 10 are smaller than 50  
80, 90, 60, 70 are greater than 50
```

The left side and right side may still be unsorted, but the pivot itself is correct.

This is the key idea of Phase 5.

7. What Is the Pivot?

The pivot is the value we choose to place in its correct sorted position.

In this curriculum's version of Quick Sort, the pivot is chosen as the **first element** of the current array section.

Example:

```
50 70 60 90 40 80 10 20 30 ∞
```

The first element is:

```
50
```

So the pivot is:

```
50
```

In code:

```
int pivot = A[low];
```

If `low = 0`, then:

```
pivot = A[0];
```

So the pivot is the first value in that section.

8. Why Is There an Infinity Marker?

In the example, the array is written like this:

```
50 70 60 90 40 80 10 20 30 ∞
```

The `∞` symbol means infinity.

In the C code, infinity is represented using:

```
INT32_MAX
```


`INT32_MAX` is a very large integer value.

It acts like a wall at the end of the array.

Why do we need this wall?

The pointer `i` moves from left to right while values are less than or equal to the pivot.

This code does that:

```
do {  
    i++;  
} while (A[i] <= pivot);
```

If the pivot is very large, `i` might keep moving right.

Without a safe stopping value at the end, `i` might go outside the array.

That would be dangerous.

So the infinity marker gives `i` a safe place to stop.

Simple meaning

The infinity marker means:

Stop here. Do not go beyond the array.

9. Important Variables in Phase 5

Variable	Simple meaning
<code>A[]</code>	The array
<code>low</code>	Starting index of the current array section
<code>high</code>	Ending boundary index, pointing to the infinity marker in this version
<code>pivot</code>	The selected value we want to place correctly
<code>i</code>	Moves from left to right
<code>j</code>	Moves from right to left
<code>temp</code>	Temporary variable used in swap

10. The Two Pointers: `i` and `j`

Partitioning uses two pointers or index variables:

`i` and `j`

They move toward each other.

Meaning of `i`

`i` starts near the left side.

It moves from left to right.

It is looking for a value that is **greater than the pivot**.

Why?

Because a value greater than the pivot does not belong on the left side.

So `i` says:

I am searching for a value that is too big to stay on the left.

Meaning of `j`

`j` starts near the right side.

It moves from right to left.

It is looking for a value that is **less than or equal to the pivot**.

Why?

Because a value less than or equal to the pivot can belong on the left side.

So `j` says:

I am searching for a value that is small enough to move to the left.

11. Pointer Movement Table

Pointer	Starts from	Moves	Stops when it finds
i	Left side	Left to right	A value greater than pivot
j	Right side	Right to left	A value less than or equal to pivot

In code:

```
do {  
    i++;  
} while (A[i] <= pivot);
```

This means:

Move **i** right while values are less than or equal to the pivot. Stop when a value is greater than the pivot.

And:

```
do {  
    j--;  
} while (A[j] > pivot);
```

This means:

Move **j** left while values are greater than the pivot. Stop when a value is less than or equal to the pivot.

12. What Happens When **i** and **j** Stop?

When **i** stops, it has found a value that is too large for the left side.

When **j** stops, it has found a value that is small enough for the left side.

So if **i < j**, we swap them.

Example:

```
50 70 60 90 40 80 10 20 30 ∞
```

Pivot:

50

`i` stops at `70` because `70 > 50`.

`j` stops at `30` because `30 <= 50`.

`70` is on the wrong side.

`30` is also on the wrong side.

So we swap them:

50 30 60 90 40 80 10 20 70 ∞

13. When Does Partitioning Stop?

Partitioning continues while:

`i < j`

That means the pointers have not crossed yet.

When `i` becomes greater than or equal to `j`, the pointers have crossed.

At that moment, scanning stops.

Then we swap the pivot with `A[j]`.

In code:

```
swap(&A[low], &A[j]);
```

This places the pivot into its final sorted position.

14. Why Do We Swap the Pivot With `A[j]` ?

This is very important.

In this partition method, after `i` and `j` cross, `j` points to the correct final position of the pivot.

So we swap:

pivot with `A[j]`

Not with `A[i]`.

Example before final pivot swap:

50 30 20 10 40 80 90 60 70 ∞

Pivot is 50.

`j` stops at 40.

So we swap 50 and 40 :

40 30 20 10 50 80 90 60 70 ∞

Now 50 is in its final position.

15. Full Dry Run of Phase 5 Partitioning

Let us partition this array:

50 70 60 90 40 80 10 20 30 ∞

We want to place the pivot 50 in its final sorted position.

Initial State

Array:

```
50 70 60 90 40 80 10 20 30 ∞
```

Pivot:

```
50
```

Initial pointers:

```
i = low  
j = high
```

In this example:

```
i starts at index 0  
j starts at index 9, the infinity marker position
```

Step 1: Move `i` and `j`

`i` moves right.

It stops at `70` because:

```
70 > 50
```

`j` moves left.

It stops at `30` because:

```
30 <= 50
```

Now:

```
i points to 70  
j points to 30
```

Since `i < j`, swap `70` and `30`.

Before swap:

50 70 60 90 40 80 10 20 30 ∞

After swap:

50 30 60 90 40 80 10 20 70 ∞

Step 2: Move **i** and **j** Again

Current array:

50 30 60 90 40 80 10 20 70 ∞

i moves right.

It skips **30** because:

$30 \leq 50$

It stops at **60** because:

$60 > 50$

j moves left.

It skips **70** because:

$70 > 50$

It stops at **20** because:

$20 \leq 50$

Now:

i points to 60
j points to 20

Since $i < j$, swap 60 and 20.

Before swap:

50 30 60 90 40 80 10 20 70 ∞

After swap:

50 30 20 90 40 80 10 60 70 ∞

Step 3: Move i and j Again

Current array:

50 30 20 90 40 80 10 60 70 ∞

i moves right.

It skips 20 because:

$20 \leq 50$

It stops at 90 because:

$90 > 50$

j moves left.

It skips 60 because:

$60 > 50$

It stops at 10 because:


```
10 <= 50
```

Now:

```
i points to 90  
j points to 10
```

Since `i < j`, swap `90` and `10`.

Before swap:

```
50 30 20 90 40 80 10 60 70 ∞
```

After swap:

```
50 30 20 10 40 80 90 60 70 ∞
```

Step 4: Move `i` and `j` Again

Current array:

```
50 30 20 10 40 80 90 60 70 ∞
```

`i` moves right.

It skips `40` because:

```
40 <= 50
```

It stops at `80` because:

```
80 > 50
```

`j` moves left.

It stops at `40` because:

```
40 <= 50
```

Now:

```
i points to 80  
j points to 40
```

At this point, `i` has crossed `j`.

So we do not swap `i` and `j` anymore.

The scanning part is finished.

Step 5: Final Pivot Swap

Before final pivot swap:

```
50 30 20 10 40 80 90 60 70 ∞
```

The pivot is:

```
50
```

`j` points to:

```
40
```

So we swap the pivot `50` with `A[j]`, which is `40`.

After final pivot swap:

```
40 30 20 10 50 80 90 60 70 ∞
```

Now the pivot `50` is fixed.

Ignoring the infinity marker, the partitioned array is:

40 30 20 10 50 80 90 60 70

We can show it like this:

40 30 20 10 | 50 | 80 90 60 70

16. Dry Run Summary

Original array:

50 70 60 90 40 80 10 20 30 ∞

After swapping 70 and 30 :

50 30 60 90 40 80 10 20 70 ∞

After swapping 60 and 20 :

50 30 20 90 40 80 10 60 70 ∞

After swapping 90 and 10 :

50 30 20 10 40 80 90 60 70 ∞

After swapping pivot 50 with 40 :

40 30 20 10 50 80 90 60 70 ∞

Final partition result without infinity:

40 30 20 10 50 80 90 60 70

Pivot final position:

index 4

17. Important Rule After Partitioning

After partitioning:

left side | pivot | right side

The rule is:

left side values are smaller than pivot
right side values are greater than pivot

Example:

40 30 20 10 | 50 | 80 90 60 70

Left side:

40 30 20 10

All are smaller than 50.

Right side:

80 90 60 70

All are greater than 50.

Therefore, 50 is in its correct final sorted position.

18. Partitioning Does Not Fully Sort the Array

This is one of the most important beginner points.

After partitioning:

40 30 20 10 | 50 | 80 90 60 70

The pivot is fixed.

But the left side is not sorted:

40 30 20 10

Correct sorted order would be:

10 20 30 40

The right side is also not sorted:

80 90 60 70

Correct sorted order would be:

60 70 80 90

So partitioning alone does not complete the sorting.

It only prepares the array for the full Quick Sort algorithm.

In Phase 6, recursion will be used to sort the left and right sides.

19. Phase 5 Pseudocode

Simple version:

```
choose the first element as pivot
move i from left to right until a value greater than pivot is found
move j from right to left until a value less than or equal to pivot is found
if i is still before j, swap A[i] and A[j]
repeat until i and j cross
swap pivot with A[j]
return j as the pivot's final position
```

Detailed version:

```
Partition(A, low, high):
    pivot = A[low]
    i = low
    j = high

    repeat:
        repeat:
            i = i + 1
            while A[i] <= pivot

        repeat:
            j = j - 1
            while A[j] > pivot

        if i < j:
            swap A[i] and A[j]

    while i < j

    swap A[low] and A[j]
    return j
```

20. Partition Function in C

```
#include <stdio.h>
#include <stdint.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int Partition(int A[], int low, int high) {
    int pivot = A[low];
    int i = low;
    int j = high;

    do {
        do {
            i++;
        } while (A[i] <= pivot);
```

```

        do {
            j--;
        } while (A[j] > pivot);

        if (i < j) {
            swap(&A[i], &A[j]);
        }

    } while (i < j);

    swap(&A[low], &A[j]);

    return j;
}

```

21. Complete Phase 5 Program in C

```

#include <stdio.h>
#include <stdint.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int Partition(int A[], int low, int high) {
    int pivot = A[low];
    int i = low;
    int j = high;

    do {
        do {
            i++;
        } while (A[i] <= pivot);

        do {
            j--;
        } while (A[j] > pivot);

        if (i < j) {
            swap(&A[i], &A[j]);
        }
    }
}

```

```

    } while (i < j);

    swap(&A[low], &A[j]);

    return j;
}

int main() {
    int A[] = {50, 70, 60, 90, 40, 80, 10, 20, 30, INT32_MAX};
    int n = sizeof(A) / sizeof(A[0]);

    int pivotIndex = Partition(A, 0, n - 1);

    printf("Pivot final index: %d\n", pivotIndex);

    for (int i = 0; i < n - 1; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}

```

22. Expected Output

```

Pivot final index: 4
40 30 20 10 50 80 90 60 70

```

The infinity marker is not printed because it is only used as a boundary value.

That is why the print loop uses:

```

for (int i = 0; i < n - 1; i++)

```

It prints all real values except `INT32_MAX`.

23. Explanation of the Complete Program

The program has three main parts:

1. `swap()` function
2. `Partition()` function
3. `main()` function

24. The `swap()` Function

```
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

This function swaps two integer values.

It uses pointers because it must change the real values inside the array.

Meaning:

Code	Meaning
<code>int *x</code>	<code>x</code> is a pointer to an integer
<code>int *y</code>	<code>y</code> is a pointer to an integer
<code>*x</code>	Value stored at the address inside <code>x</code>
<code>*y</code>	Value stored at the address inside <code>y</code>
<code>temp</code>	Temporary variable used to protect one value

When the partition function calls `swap`, it calls it like this:

```
swap(&A[i], &A[j]);
```

or:

```
swap(&A[low], &A[j]);
```

The `&` means address of.

So this gives the addresses of the two array elements.

25. The Partition() Function Header

```
int Partition(int A[], int low, int high)
```

This function receives:

Parameter	Meaning
A[]	The array
low	Starting index of the section being partitioned
high	Ending boundary index, pointing to the infinity marker in this version

The return type is `int` because the function returns the final index of the pivot.

26. Choosing the Pivot

```
int pivot = A[low];
```

This chooses the first element as the pivot.

For the array:

```
50 70 60 90 40 80 10 20 30 ∞
```

`low = 0`, so:

```
pivot = A[0];
```

Therefore:

```
pivot = 50
```

27. Initializing `i` and `j`

```
int i = low;  
int j = high;
```

This sets the starting positions of the two pointers.

For the example:

```
low = 0  
high = 9
```

So:

```
i = 0  
j = 9
```

`j` starts at the infinity marker position.

28. Moving `i`

```
do {  
    i++;  
} while (A[i] <= pivot);
```

This moves `i` to the right.

It keeps moving while values are less than or equal to the pivot.

It stops when it finds a value greater than the pivot.

Simple meaning:

Move right until you find something too big for the left side.

Example:

```
50 70 60 90 40 80 10 20 30 ∞
```

Pivot:

50

`i` moves from 50 to 70.

It stops at 70 because:

70 > 50

29. Moving `j`

```
do {  
    j--;  
} while (A[j] > pivot);
```

This moves `j` to the left.

It keeps moving while values are greater than the pivot.

It stops when it finds a value less than or equal to the pivot.

Simple meaning:

Move left until you find something small enough for the left side.

Example:

50 70 60 90 40 80 10 20 30 ∞

Pivot:

50

`j` moves from infinity to 30.

It stops at 30 because:

30 <= 50

30. Swapping `A[i]` and `A[j]`

```
if (i < j) {  
    swap(&A[i], &A[j]);  
}
```

This means:

If `i` and `j` have not crossed, swap the two wrong-side values.

Example:

`i` points to 70.

`j` points to 30.

Since 70 belongs on the right and 30 belongs on the left, they are swapped.

Before:

50 70 60 90 40 80 10 20 30 ∞

After:

50 30 60 90 40 80 10 20 70 ∞

31. The Outer `do-while` Loop

```
do {  
    ...  
} while (i < j);
```

This repeats the pointer movement and swapping while `i` is before `j`.

Once `i` crosses `j`, the loop stops.

Simple meaning:

Keep scanning and swapping until the two pointers cross.

32. Final Pivot Swap

```
swap(&A[low], &A[j]);
```

After `i` and `j` cross, `j` points to the final position of the pivot.

So we swap the pivot with `A[j]`.

Example:

Before final swap:

```
50 30 20 10 40 80 90 60 70 ∞
```

Pivot:

```
50
```

`A[j]`:

```
40
```

After final swap:

```
40 30 20 10 50 80 90 60 70 ∞
```

Now the pivot is fixed.

33. Returning `j`

```
return j;
```

The function returns `j` because `j` is the final index of the pivot.

In the example:

pivot 50 ends at index 4

So the function returns:

4

This returned index will be very important in Phase 6, when Quick Sort uses recursion.

34. The `main()` Function

```
int main() {
    int A[] = {50, 70, 60, 90, 40, 80, 10, 20, 30, INT32_MAX};
    int n = sizeof(A) / sizeof(A[0]);

    int pivotIndex = Partition(A, 0, n - 1);

    printf("Pivot final index: %d\n", pivotIndex);

    for (int i = 0; i < n - 1; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}
```

This function:

1. Creates an array.
 2. Adds `INT32_MAX` as the infinity marker.
 3. Calculates the number of elements.
 4. Calls the `Partition()` function.
 5. Prints the pivot's final index.
 6. Prints the real array values, excluding the infinity marker.
-

35. Meaning of `sizeof(A) / sizeof(A[0])`

```
int n = sizeof(A) / sizeof(A[0]);
```

This finds the number of elements in the array.

Example:

```
int A[] = {50, 70, 60, 90, 40, 80, 10, 20, 30, INT32_MAX};
```

There are 10 values including `INT32_MAX`.

So:

```
n = 10
```

The real sortable values are only 9 values.

The last value is only the infinity marker.

36. Why We Print Only `n - 1` Values

The array contains:

```
50 70 60 90 40 80 10 20 30 INT32_MAX
```

`INT32_MAX` is not part of the real data.

It is only a boundary marker.

So we print only the first `n - 1` values:

```
for (int i = 0; i < n - 1; i++) {  
    printf("%d ", A[i]);  
}
```

This avoids printing the infinity marker.

37. Complexity of One Partition Step

Partitioning scans the array section using `i` and `j`.

Each pointer moves through part of the array.

Together, they process the section in linear time.

So one partition step takes:

$O(n)$

This means:

If the section has `n` elements, partitioning does work proportional to `n`.

38. Space Complexity of Partitioning

The partition function does not create another large array.

It only uses a few variables:

`pivot, i, j, temp`

So the extra space for partitioning is:

$O(1)$

This means constant extra space.

39. Is Partitioning In-Place?

Yes.

Partitioning is in-place because it rearranges values inside the original array.

It does not create a second array to store the result.

It only swaps values inside the same array.

So partitioning is:

In-place

40. Is Partitioning a Full Sorting Algorithm?

No.

Partitioning alone is not a full sorting algorithm.

It only places one pivot into its correct position.

Example:

40 30 20 10 | 50 | 80 90 60 70

Only 50 is fixed.

The left side and right side still need sorting.

Full Quick Sort will sort those parts later using recursion.

41. Is This Partitioning Stable?

Usually, no.

This partitioning method uses swaps.

Swaps can move equal values around and may change their original relative order.

So this partitioning method should not be treated as stable.

Full Quick Sort is usually not stable in its common in-place form.

For Phase 5, the most important point is:

Partitioning focuses on placing the pivot correctly, not preserving duplicate order.

42. Why Phase 5 Does Not Teach Full Quick Sort Yet

Quick Sort has two major ideas:

1. Partitioning
2. Recursion

Partitioning is the heart of Quick Sort.

If you do not understand partitioning, Quick Sort recursion will feel confusing.

That is why Phase 5 teaches only partitioning.

Phase 6 will use the returned pivot index to sort the left and right parts recursively.

Example after Phase 5 partitioning:

```
40 30 20 10 | 50 | 80 90 60 70
```

Phase 6 will sort:

```
40 30 20 10
```

and:

```
80 90 60 70
```

Then the full array will become sorted.

43. Beginner Analogy for Partitioning

Imagine you are organizing students by height around one selected student.

The selected student is the pivot.

You want:

```
shorter students | selected student | taller students
```

You do not need to fully arrange the shorter students yet.

You do not need to fully arrange the taller students yet.

You only need to put the selected student in the correct middle position.

That is partitioning.

44. Memory Trick for Phase 5

Remember this sentence:

Pivot in the middle, smaller left, greater right.

Another useful memory trick:

Choose pivot, scan from both sides, swap wrong-side values, place pivot at `j`.

45. Common Beginner Mistakes

Mistake 1: Thinking Partitioning Fully Sorts the Array

Wrong idea:

After partitioning, the whole array is sorted.

Correct idea:

After partitioning, only the pivot is in its correct position.

Example:

```
40 30 20 10 | 50 | 80 90 60 70
```

The pivot `50` is correct, but the two sides are not fully sorted.

Mistake 2: Forgetting the Infinity Marker

This exact code uses:

```
INT32_MAX
```

at the end of the array.

If you forget the infinity marker, the `i` pointer may move outside the array.

That can cause incorrect behavior or a crash.

Mistake 3: Printing the Infinity Marker

The infinity marker is not real data.

Wrong idea:

```
for (int i = 0; i < n; i++)
```

This prints `INT32_MAX` too.

Better for this example:

```
for (int i = 0; i < n - 1; i++)
```

This prints only the real array values.

Mistake 4: Swapping the Pivot With `A[i]`

Wrong idea:

```
swap(&A[low], &A[i]);
```

Correct idea in this version:

```
swap(&A[low], &A[j]);
```

In this partition method, `j` gives the final pivot position.

Mistake 5: Changing the Comparison Signs Carelessly

This version uses:

```
while (A[i] <= pivot)
```

and:

```
while (A[j] > pivot)
```

Changing these signs can change how the partition behaves.

Partition methods must be written carefully.

Do not mix comparison rules from different textbooks unless you understand the partition scheme.

Mistake 6: Forgetting That `high` Points to the Infinity Marker

In this example:

```
int A[] = {50, 70, 60, 90, 40, 80, 10, 20, 30, INT32_MAX};
```

The last index is the infinity marker.

The function call is:

```
Partition(A, 0, n - 1);
```

Here, `n - 1` points to `INT32_MAX`.

This is different from many other array algorithms where `high` points to the last real element.

Mistake 7: Confusing `i` and `j`

Remember:

```
i moves left to right  
j moves right to left
```

`i` looks for a value greater than the pivot.

`j` looks for a value less than or equal to the pivot.

At the end, the pivot swaps with `A[j]`.

Mistake 8: Forgetting to Return `j`

The partition function should return the final pivot index.

Correct:

```
return j;
```

This value is needed later by Quick Sort recursion.

46. Phase 5 Key Vocabulary

Term	Simple meaning
Quick Sort	A sorting algorithm based on partitioning and recursion
Partitioning	Placing a pivot so smaller values are left and greater values are right
Pivot	The selected value we place in its final position
Final sorted position	A position where all smaller values are left and all greater values are right
<code>low</code>	Starting index of the current section
<code>high</code>	Ending boundary index in this code
<code>i</code>	Pointer moving from left to right
<code>j</code>	Pointer moving from right to left
Infinity marker	A very large boundary value used to stop <code>i</code> safely
<code>INT32_MAX</code>	A large integer value used as infinity in C
Crossing	When <code>i</code> moves past or reaches <code>j</code>
In-place	Works inside the original array
<code>O(n)</code>	Linear time
<code>O(1)</code> space	Constant extra memory

47. Phase 5 Evaluation Checklist

When studying Quick Sort partitioning, ask these questions.

Pivot

Which value is chosen as the pivot?

In this version:

The first value is chosen as pivot.

Left and Right Rule

After partitioning, what should be true?

Smaller values should be on the left.
Greater values should be on the right.
The pivot should be between them.

Pointers

What does `i` do?

It moves from left to right and stops at a value greater than the pivot.

What does `j` do?

It moves from right to left and stops at a value less than or equal to the pivot.

Swapping

When do we swap `A[i]` and `A[j]`?

When $i < j$.

When do we swap the pivot?

After i and j cross, swap pivot with $A[j]$.

Sorting Result

Does partitioning fully sort the array?

No.

What does it fix?

Only the pivot.

Complexity

What is the time complexity of one partition step?

$O(n)$

What is the extra space complexity?

$O(1)$

48. Complete Phase 5 Program With Comments

```
#include <stdio.h>
#include <stdint.h>

// Function to swap two integer values using pointers
void swap(int *x, int *y) {
    int temp = *x;
```

```

    *x = *y;
    *y = temp;
}

// Partition function for Quick Sort
int Partition(int A[], int low, int high) {
    // Choose the first element as pivot
    int pivot = A[low];

    // i starts from the left side
    int i = low;

    // j starts from the right boundary
    int j = high;

    do {
        // Move i right until A[i] is greater than pivot
        do {
            i++;
        } while (A[i] <= pivot);

        // Move j left until A[j] is less than or equal to pivot
        do {
            j--;
        } while (A[j] > pivot);

        // If i and j have not crossed, swap wrong-side values
        if (i < j) {
            swap(&A[i], &A[j]);
        }

    } while (i < j);

    // Put the pivot into its final position
    swap(&A[low], &A[j]);

    // Return the pivot's final index
    return j;
}

int main() {
    // INT32_MAX acts as the infinity marker
    int A[] = {50, 70, 60, 90, 40, 80, 10, 20, 30, INT32_MAX};

    // Find number of elements, including infinity marker
    int n = sizeof(A) / sizeof(A[0]);

    // Partition the array

```

```

int pivotIndex = Partition(A, 0, n - 1);

// Print pivot's final index
printf("Pivot final index: %d\n", pivotIndex);

// Print real values only, not the infinity marker
for (int i = 0; i < n - 1; i++) {
    printf("%d ", A[i]);
}

printf("\n");
return 0;
}

```

49. Phase 5 Summary

Phase 5 teaches Quick Sort partitioning.

The main idea is:

Choose a pivot and place it in its final sorted position.

Important points:

- Quick Sort is split into partitioning and recursion.
- Phase 5 teaches only partitioning.
- Partitioning is the heart of Quick Sort.
- The pivot is the selected value.
- In this version, the first element is chosen as the pivot.
- The pivot should end up between smaller values and greater values.
- `i` moves from left to right.
- `i` stops at a value greater than the pivot.
- `j` moves from right to left.
- `j` stops at a value less than or equal to the pivot.
- If `i < j`, swap `A[i]` and `A[j]`.
- When `i` and `j` cross, swap the pivot with `A[j]`.
- The function returns `j`, the final pivot index.
- The infinity marker `INT32_MAX` is used as a safe boundary in this code.
- Partitioning does not fully sort the array.
- Partitioning only fixes the pivot.
- One partition step takes $O(n)$ time.
- Partitioning uses $O(1)$ extra space.
- Partitioning is in-place.
- Full Quick Sort will be completed in the next phase using recursion.

50. What Comes Next

After Phase 5, you are ready for Phase 6: Quick Sort Recursion and Complexity.

In Phase 6, you will learn how Quick Sort uses the pivot index returned by `Partition()`.

After one partition, we get:

```
40 30 20 10 | 50 | 80 90 60 70
```

Phase 6 will recursively sort the left side:

```
40 30 20 10
```

and the right side:

```
80 90 60 70
```

That is how full Quick Sort completes the sorting process.

The simple idea is:

Partition, then sort the left part and the right part.

But for now, the most important Phase 5 idea is:

Pivot in the middle, smaller left, greater right.